Intro to SQL and Bottle for Building AI Apps in Python (Full Day)

Table of Contents

Setup....3
What is a Relational Databases....4
RDMS - Relational Database Management Systems....4
Schema....4
JOINS....4
SQLite and Security....5
SQLite Shell....5
SQLite Commands....5
SQL Syntax and Styling....7
Delimiter....7
Capitalization....7
Lines....7
Create a Database....7
Create/Alter/Drop a Table....8
Data Types in SQLite....8
Primary Keys....8
Timestamps....9
Strict Mode....9
Alter....9
Drop....9
Insert....10
Select....10
Where....11
Is....11
Conditionals....11
Like and Wildcards....11
Order By....11
Limit....12
AND, OR Operators....12
Update....13
Delete....13
Join....13
Inner Join....14
Left Join....15
SQL Functions....15
LAB – Create Database....17

Create Database, Tables and Insert Records....17
Selecting Records....18
Selecting Records with Left Join....18
Update Records....19
Delete Records....19
Python with SQLite....20
Selecting Records....20
Parameter Placeholders....21
Create Table....22
Insert....22
Update....22
Python and Ollama with SQLite....23
LAB – AI Caching....23
LAB - Rules....25
What is Bottle....27
Why Use Bottle....27
Architecture....27
Run, Route and Return....28
Routes....28
Static Routes....29
Dynamic Routes....29
Methods – POST and GET....30
POST....30
GET....31
Static Files....32
Templates....33
Template Variables....34
Template Controls....35
LAB - Basic LLM App....36
LAB - AI Web App Using Templates....37
LAB - AI Web App with SQL Caching....38
Deploying A Bottle Web App....40
Install Requirements to Run Bottle and Verify....40
Install Gunicorn....42
Create Linux Service....43
Configure NGINX to Route to Gunicorn....43
Set DNS to Point to VPS....45

This class teaches the basic concepts of using a SQL database as a record store for LLM's and other AI models. You can create reports of requests made, memory to add to context windows, cacheing mechanisms and more.

It also teaches how to create and deploy web applications using the Bottle Web App Framework

Setup

For this class you'll need to install:

- Python3
- PIP
- Ollama
- VSCode
- VSCode Python Extension

To create a web app that is internet accessible you will need accounts with:

- DigitalOcean for a VPS
- CloudFlare for DNS

SQLite

What is a Relational Databases

Relational Databases are reliable systems for storing records. They are bound by a set of rules for transactions called ACID

- A — Atomicity: All or nothing.
  - If a transaction has 5 steps, either all 5 happen, or none happen.
- C — Consistency: Rules stay true.
  - After a transaction, the database is still valid (constraints/relationships aren't broken).
- I — Isolation: Transactions don't step on each other.
  - Two users running transactions at the same time shouldn't produce weird mixed results.
- D — Durability: Once committed, it stays saved.
  - If the DB says "commit succeeded," the data survives crashes/power loss (within the DB's guarantees).

Relational Databases are good for when transactions cannot be lost such as for logging payments.

RDMS - Relational Database Management Systems

RDMS are the server software that provides the database service. Popular RDMS's include MySQL, Postgress, MariaDB, MSSQL and SQLite.

SQL (Structured Query Language) is generally the same in the RDMS's, but their capabilities are different. Such a SQLite is easy and fast, but has no security. MySQL provides robust security, but requires more administration.

Schema

The schema is the design of the database tables. You define the columns which are the values for the records along with data types. You design and implement the schema and then interact with the database based on that. Modifying the schema can be a dangerous process.

JOINS

In relational databases specific data is help within individual tables and then records are linked using a JOIN statement. So you have one table for PARTS and one for VENDORS. The PARTS table would have the part name, description, price and vendor ID. You will then have the VENDOR table contain

the name of the vendor, their address, contact information, and ID. You can then do a SQL statement where you link the parts with the vendors using a JOIN statement.

SQLite and Security

Doesn't exist...

SQLite Shell

SQLite has a command line shell like most tools do. You use the command **sqlite3** to open the shell.

When you use sqlite3 without calling a database file then a temporary database will be created. If you do not save that database then when you exit all of your work will be detroyed.

```
sqlite3
```

```
eli@osx ~ % sqlite3
SQLite version 3.51.0 2025-06-12 13:14:41
Enter ".help" for usage hints.
Connected to a transient in-memory database.
Use ".open FILENAME" to reopen on a persistent database.
sqlite>
```

To save your database use the **.backup** command with a database name. Note that the .db file extension is only there for your knowledge.

```
sqlite> .backup test.db
```

SQLite Commands

SQLite commands are used to administer SQLite. These are different than SQL statements. Every database system will have it's own commands and ways of being managed.

In SQLite you use a period '.' before a command and then the command. You do not use a semi colon ';' as a demlimiter.

Exit

```
.exit
```

Help

```
.help
```

List Tables

```
.tables
```

```
sqlite> .tables
log      student  test
```

Show Schema for Table

The .schema command provides the statement that was used to create a table.

```
.schema TABLENAME
```

```
sqlite> .schema log
CREATE TABLE log(
problem TEXT,
timestamp TEXT DEFAULT CURRENT_TIMESTAMP);
```

Headers

```
.headers on
```

```
sqlite> select * from student;
1|bob|12
sqlite> .headers on
sqlite> select * from student;
id|name|age
1|bob|12
```

Mode

```
.mode column
```

```
sqlite> select * from student;
id|name|age
1|bob|12
sqlite> .mode column
sqlite> select * from student;
id  name  age
--  ----  ---
1   bob   12
```

SQL Syntax and Styling

Delimiter

In SQL the delimiter is the semicolon ';'. Until you use a semi colon the SQL statement will not run.

Capitalization

Capitalization in SQL does not matter for statements. Generally SQL statements are written in capital letters such as: **SELECT * from TABLE;** but this is not required. **select * from table;** works perfectly. In statements capitalization is about Styling. Just like in HTML capitalization used to be the standard, but for many it has fallen out of favor.

When dealing with values in a table capitalization does matter. BOB, Bob, bob will all be different values in a table.

Lines

When looking at documentation for SQL you will see statements on multiple lines. This can happen becasue the statement does not run until you add the semi colon.

```
create TABLE table_name(
      name TEXT,
      age INTEGER,
      size TEXT);
```

This is not necessary, and with using the up/down arrow keys for rerunning commands can be tediuos.

```
create TABLE table_name(name TEXT,age INTEGER,size TEXT);
```

Create a Database

To create a database you simply call it. When you call a database name SQLite will either open an existing database of that name, or will create one with that name.

Make sure to use the right file path to the database you want to open. You can end up with numerous databases with the same name scattered around your computer.

We use the .db extension for databases simply for usability so we know which files in a folers are database files.

```
sqlite3 class.db
```

```
SQLite version 3.51.0 2025-06-12 13:14:41
Enter ".help" for usage hints.
sqlite>
```

Create/Alter/Drop a Table

When creating a table in SQLite you do not have to specify data types, or a primary key.

A column named ROW_ID is automatically created and auto increments by one when a new record is created.

NOTE: If you delete a record the ROW_ID number may be reused for a new record.

```
create table student(name,age);
```

Data Types in SQLite

https://www.sqlite.org/datatype3.html

- **NULL**. The value is a NULL value.
- **INTEGER**. The value is a signed integer, stored in 0, 1, 2, 3, 4, 6, or 8 bytes depending on the magnitude of the value.
- **REAL**. The value is a floating point value, stored as an 8-byte IEEE floating point number.
- **TEXT**. The value is a text string, stored using the database encoding (UTF-8, UTF-16BE or UTF-16LE).
- **BLOB**. The value is a blob of data, stored exactly as it was input.

SQLite Data Typing is very loose compared to other database systems. Officially there are 5 data types, but generally data types for columns is used just for your information and has no effect on the system itself.

```
sqlite> create table student(
        name TEXT,
        age INTEGER);
```

This also works

```
sqlite> create table student(
        name NOTADATATYPE,
        age GARBAGENUMBER);
```

Primary Keys

Every record in a database has to be unique in some way. By default this is done with the ROW_ID column that auto increments by one with each new record. The issue with this is that the ROW_IP number may be reused if a record is deleted. So it your code uses the ROW_ID to point to a specific record through normal use it could end up pointing to the wrong record.

An Auto Incrementing Primary Key solves this problem. It auto increments by one with each new record that is created, and will not reuse the number if a record is deleted.

```
sqlite> create table student(
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT,
        age INTEGER);
```

Timestamps

You can add a timestamp column by setting the default value to CURRENT_TIMESTAMP

```
sqlite> create table log(
        problem TEXT,
        timestamp TEXT DEFAULT CURRENT_TIMESTAMP);
```

```
sqlite> select * from log;
problem      timestamp
-----------  -------------------
router down  2026-02-13 15:55:40
ping failed  2026-02-13 15:55:47
```

Strict Mode

Strict Mode enforces data types in SQLite.

```
sqlite> create table student(
        name TEXT,
        age INT)strict;
```

```
sqlite> create table student(name WORDS, age NUMBER)strict;
Parse error: unknown datatype for student.name: "WORDS"
```

Alter

You can modify tables once they have been created. Be very careful doing this. It is a “destructive” action which means you might lose data if you do something improperly

```
ALTER TABLE old_table_name RENAME TO new_table_name;
```

```
ALTER TABLE users ADD COLUMN age INTEGER;
```

```
ALTER TABLE users RENAME COLUMN fullname TO name;
```

```
ALTER TABLE users DROP COLUMN age;
```

Drop

To delete a table you drop it. This command is permanent and instant.

```
drop table student;
```

Insert

Insert statements add records to your table. You state the table and columns you want data added to, and then you state the values the columns should be set to.

```
insert into student(name,age)values('sue',15);
```

Select

Select statements query records from tables in your database. You state the columns you want data from and then the table to be queried.

* means all columns other than ROWID

```
sqlite> select * from student;
id  name  age
--  ----  ---
1   bob
2   sue   22
3   tim   2
4   pam   13
5   fred  15
6   pete  8
```

```
sqlite> select name, age from student;
name  age
----  ---
bob
sue   22
tim   2
pam   13
fred  15
pete  8
```

To access the ROWID value you have to directly call it.

```
sqlite> select rowid, id, name, age from student;
id  id  name  age
--  --  ----  ---
1   1   bob
2   2   sue   22
3   3   tim   2
4   4   pam   13
5   5   fred  15
6   6   pete  8
```

Where

The where clause allows you to select records based on specific criteria.

Is

You use the is clause to select any records that have a specific value.

```
sqlite> select * from student where name is 'sue';
id  name  age
--  ----  ---
2   sue   22
7   sue   15
```

Conditionals

For columns with number values you can use >,>=,<,<= conditionals to find records.

```
sqlite> select * from student where age > 14;
id  name  age
--  ----  ---
2   sue   22
5   fred  15
```

Like and Wildcards

With the like clause you can use wildcards in queries to act as a search function.

- % means any value any number of iterations
- _ means any character for exactly one iteration

```
sqlite> select * from student where name like '%e%';
id  name  age
--  ----  ---
2   sue   22
5   fred  15
6   pete  8
```

Order By

You can order the results of a query based on a column. You use the order by clause and then the column at the end of the statement. By default this will sort ascending.

```
sqlite> select * from student order by age;
id  name  age
--  ----  ---
1   bob
3   tim   2
6   pete  8
4   pam   13
5   fred  15
7   sue   15
2   sue   22
```

To sort descending add desc at end of the statement.

```
sqlite> select * from student order by age desc;
id  name  age
--  ----  ---
2   sue   22
5   fred  15
7   sue   15
4   pam   13
6   pete  8
3   tim   2
1   bob
```

Limit

You can limit how many results are returned by a query. You add the limit clause to the end of the statement with the number of records to be returned.

```
sqlite> select * from student order by age desc limit 3;
id  name  age
--  ----  ---
2   sue   22
5   fred  15
7   sue   15
```

AND, OR Operators

You can test for multiple conditions by using the AND or OR operators.

```
sqlite> select * from student where age < 20 and name is 'sue';
id  name  age
--  ----  ---
7   sue   15
```

```
sqlite> select * from student where age < 18 and age > 8;
id  name  age
--  ----  ---
4   pam   13
5   fred  15
7   sue   15
```

```
sqlite> select * from student where age < 10 or name is 'sue' ;
id  name  age
--  ----  ---
2   sue   22
3   tim   2
6   pete  8
7   sue   15
```

Update

You change records by using the update statement. You use the set keyword to update specific columns, and can update more than one at once.

You then use the where clause to state which records should be updated. This is why a unique identifier is important so that you have a way to identify a specific record. When update runs it will modify all records that match the where clause.

```
sqlite> update student set name = 'susan' where id = 2;
```

```
sqlite> update student set name = 'susan', age = 23 where id = 2;
```

```
sqlite> select * from student;
id  name   age
--  -----  ---
1   bob
2   susan  23
3   tim    2
4   pam    13
5   fred   15
6   pete   8
7   sue    15
```

Delete

The delete statement will delete records based on the where clause. This is an immediate and permanent action.

```
sqlite> delete from student where id = 4;
```

```
sqlite> select * from student;
id  name   age
--  -----  ---
1   bob
2   susan  23
3   tim    2
5   fred   15
6   pete   8
7   sue    15
```

Join

Joins are the real power of Relational Databases. Each table should focus on only data that should be unique. You then join tables together in queries to pull out all of the information that you need.

So you may have a PART table and a VENDOR table. The PART table would containing the name, price, description, etc. The VENDOR table would contain the name of the vendor, contact information, location, etc. You could the do a JOIN statement to connect the PART and VENDOR tables together to provide the full information for a part.

Or you could have an invoicing system where you have an INVOICE table and a CUSTOMER table where you can connect the customers to the invoices.

Example Tables

We have created 2 tables for the following examples. The first is for pupils in a calss, and the second is for allergies. We will join the tables using the value under pupil.allergy and allergy.name

pupil

```
name   age  allergy
-----  ---  -------
bob    12   peanut
sue    14   sun
tim    10   none
frank  18   peanut
phil   8    peanut
tammy  8    sun
tony   11
lucy   5
```

allergy

```
sqlite> select * from allergy;
name    condition
------  ------------------
peanut  Don't give peanuts
sun     Use suntan lotion
none    Do nothing
grass   Play on cement
```

Inner Join

An inner join joins two tables based on identical values in a column. Note that only records that fully match will be shown.

```
select * from TABLE1 inner join TABLE2 on TABLE1.COLUMN = TABLE2.COLUMN;
```

```
sqlite> select * from pupil inner join allergy on pupil.allergy = allergy.name;
```

```
name   age  allergy  name    condition
-----  ---  -------  ------  ------------------
bob    12   peanut   peanut  Don't give peanuts
sue    14   sun      sun     Use suntan lotion
tim    10   none     none    Do nothing
frank  18   peanut   peanut  Don't give peanuts
phil   8    peanut   peanut  Don't give peanuts
tammy  8    sun      sun     Use suntan lotion
```

Left Join

Left joins show all records from the first table, and then adds the values for records that match from the second table.

```
select * from TABLE1 left join TABLE2 on TABLE1.COLUMN = TABLE2.COLUMN;
```

```
sqlite> select * from pupil left join allergy on pupil.allergy = allergy.name;
```

```
name   age  allergy  name    condition
-----  ---  -------  ------  ------------------
bob    12   peanut   peanut  Don't give peanuts
sue    14   sun      sun     Use suntan lotion
tim    10   none     none    Do nothing
frank  18   peanut   peanut  Don't give peanuts
phil   8    peanut   peanut  Don't give peanuts
tammy  8    sun      sun     Use suntan lotion
tony   11
lucy   5
```

SQL Functions

SQL has a number of built in functions. These can be used to quickly do basic math, or transformations.

count() - counts the number of records based on a column value

```
sqlite> select count(name) from student;
count(name)
-----------
9
```

min() - provides the minimum value in a column.

```
sqlite> select min(age) from student;
min(age)
--------
8
```

max() - provides the maximum value in a column

```
sqlite> select max(age) from student;
max(age)
--------
111
```

avg() - provides an average for the values in a column

```
sqlite> select avg(age) from student;
avg(age)
----------------
24.1111111111111
```

upper() or lower() - change the capitalization of the results for the column

```
sqlite> select upper(name), age from student;
upper(name)  age
-----------  ---
BOB          12
TIM          13
SUE          13
FRANK        17
PHIL         16
PAM          16
TIMMY        111
TYRONE       8
WILLIAM      11
```

LAB – Create Database

In this lab we are going to create a database and tables to track students and their allergies.

Create Database, Tables and Insert Records

Create a Database named class.db

```
sqlite3 class.db
SQLite version 3.51.0 2025-06-12 13:14:41
Enter ".help" for usage hints.
sqlite>
```

Create a table named STUDENT.

- name
- age
- shirt size
- allergy

```
sqlite> create table student(name,age,size,allergy);
sqlite> .schema student
CREATE TABLE student(name,age,size,allergy);
```

Create a table named ALLERGY

- name
- condition
- treatment

```
sqlite> create table allergy(name,condition,treatment);
sqlite> .schema allergy
CREATE TABLE allergy(name,condition,treatment);
```

Add 5 or more records to STUDENT table.

```
insert into student(name,age,size,allergy)values('bob',12,'large','peanut');
insert into student(name,age,size,allergy)values('tim',13,'small','sun');
insert into student(name,age,size,allergy)values('sue',13,'small','grass');
insert into student(name,age,size,allergy)values('frank',17,'small','sun');
insert into student(name,age,size,allergy)values('phil',16,'medium','sun');
insert into student(name,age,size,allergy)values('pam',16,'medium','grass');
insert into student(name,age,size)values('timmy',111,'medium');
insert into student(name,age,size,allergy)values('tyrone',8,'medium','grass');
insert into student(name,age,size)values('billy',11,'small');
```

Add 3 or more records to ALLERGY table.

```
insert into allergy(name,condition,treatment)values('grass','Allergic to grass,
keep off grass','benadryl');
insert into allergy(name,condition,treatment)values('sun','Sunburns easily','wear
sunscreen');
insert into allergy(name,condition,treatment)values('peanut','cannot eit
peanuts','call 911');
```

Selecting Records

Now that we have a database, tables and records you can start querying the tables for records.

Select all columns and records from STUDENT table

```
select * from student;
```

Select the names of students whose shirt size is small

```
select name from student where size is 'small';
```

Select the names and ages of students that are juvenile teenagers

```
select name,age from student where age <=17 and age >=13;
```

Select names who have a ‘b’ in their name

```
select name from student where name like '%b%';
```

Selecting Records with Left Join

With left join you will now connect the student and allergy tables to be able to pull out record values from each table.

Select all columns from STUDENT and ALLERGY using left join.

```
select * from student left join allergy on student.allergy = allergy.name;
```

Select students name, age and the allergy condition using a left join

```
select student.name, student.age, allergy.condition from student left join allergy
on student.allergy = allergy.name;
```

Select student name, their allergy and condition for any student aged 13

```
select student.name, student.allergy, allergy.condition from student left join
allergy on student.allergy = allergy.name where age = 13;
```

Update Records

Update a single record and change the name of that student

```
update student set name = 'william' where name = 'billy';
```

Update numerous records to change the value for shirt size from ‘medium’ to ‘med’

```
update student set size = 'med' where size = 'medium';
```

Delete Records

Now delete a single record, or multiple records using a delete statement.

```
delete from student where name = 'william';
```

Python with SQLite

Python connects to SQLite with the sqlite3 module. This is a core module that is part of Python.

You will hear of modules such as SQLAlchemy and concepts such as ORM's (Object Relational Mapper) for interacting with SQL on Python, but that is not what we'll be teaching in this class.

Selecting Records

To select and return records using Python and SQL you have to go through a few steps.

When Python gets the records it will format them into a list of tuples. You can then iterate through the list with a for loop.

- connect() - Connects to database
- cursor() - Creates a way to run commands on SQLite. Like a “cursor” for your code
- execute() - Executes your SQL statement
- fetchall() or fetchone() - You can either fetch all records or return the first that comes back. You can use fecthall() with limit in your SQL for a specific number of records
- close() - Closes the database connection.

sqlite-python-select-all.py

```
import sqlite3

conn = sqlite3.connect('class.db')
cursor = conn.cursor()
sql = 'select * from student'
result = cursor.execute(sql,)
result = result.fetchall()
conn.close()

print(result)

for x in result:
    print(x)

for x in result:
    print(f'{x[1]} \t {x[2]}')
```

```
[(1, 'bob', None), (2, 'susan', 23), (3, 'tim', 2), (5, 'fred', 15), (6, 'pete',
8), (7, 'sue', 15)]
(1, 'bob', None)
(2, 'susan', 23)
(3, 'tim', 2)
(5, 'fred', 15)
(6, 'pete', 8)
(7, 'sue', 15)
bob      None
```

```
susan    23
tim      2
fred     15
pete     8
sue      15
```

Parameter Placeholders

To prevent SQL Injection attacks you use Parameter Placeholders to pass values to the SQL statement.

You use ? In the SQL Statement, and then pass the values for the ? in the execute() function. These values are read positionally. So the first ? will use the first value in the execute() function.

The values in the execute function are passed as a tuple so even if you are only passing one value you need to provide the ‘,’ for the value to be seen as part of a tuple.

- execute(sql,(tuple,))

sqlite-python-select-value.py

```
import sqlite3

age = 13

conn = sqlite3.connect('class.db')
cursor = conn.cursor()
sql = 'select * from student where age < ?'
result = cursor.execute(sql,(age,))
result = result.fetchall()
conn.close()

print(result)
```

```
[(3, 'tim', 2), (6, 'pete', 8)]
```

sqlite-python-select-value-multiple.py

```
import sqlite3

max_age = 18
min_age = 13

conn = sqlite3.connect('class.db')
cursor = conn.cursor()
sql = 'select * from student where age < ? and age >= ?'
result = cursor.execute(sql,(max_age,min_age))
result = result.fetchall()
conn.close()

print(result)
```

```
[(5, 'fred', 15), (7, 'sue', 15)]
```

Create Table

You can both create a database and a table if they do not exist when your Python script runs.

You connect to the database, and then use CREATE TABLE IF NOT EXISTS statement to create a table if it does not exist.

```
conn = sqlite3.connect("database_name.db")
cursor = conn.cursor()
cursor.execute("""
    CREATE TABLE IF NOT EXISTS table_name (
        column1,
        column2
    );
    """)
conn.commit()
conn.close()
```

Insert

To insert records into a table it is done like the select statement, but without the fetch() function. Create the insert SQL statement with '?' placeholders, and then add the values in the execute() funtion.

```
conn = sqlite3.connect("database_name.db")
cursor = conn.cursor()
sql = 'insert into table_name(column1,column2) values(?,?)'
cursor.execute(sql,('value1','value2'))
conn.commit()
conn.close()
```

Update

To update records you use the update statement with the '?' placehoders as done previously.

```
conn = sqlite3.connect("database_name.db")
cursor = conn.cursor()
sql = 'update table_name set column1 = ?, column2 = ? where rowid = ?'
cursor.execute(sql,('value1','value2','value3'))
conn.commit()
conn.close()
```

Python and Ollama with SQLite

These labs will use Python, SQLite and Ollama to build command line AI projects. You should use the highest quality LLM that functions well on your computer to see the full results.

```
Install Ollama from Ollama.com
```

```
python3 -m pip install ollama
```

LAB – AI Caching

This lab creates a caching mechanism for AI Queries. When a request is made and the response is received from Ollama those values are saved in the database.

If you make the exact same request again the response is pulled from the database instead of being processed through Ollama.

This will work with any LLM Model. granite4:350m, granite4:3b, gpt-oss

sqlite-ollama-cache.py

```
from ollama import chat, ChatResponse
import sqlite3

def create():
    conn = sqlite3.connect("ai-cache.db")
    cursor = conn.cursor()
    cursor.execute("""
    CREATE TABLE IF NOT EXISTS cache (
        query,
        response
    );
    """)
    conn.commit()
    conn.close()

def new(query,response):
    conn = sqlite3.connect("ai-cache.db")
    cursor = conn.cursor()
    sql = 'insert into cache(query, response) values(?,?)'
    cursor.execute(sql,(query, response))
    conn.commit()
    conn.close()

def find(query):
    conn = sqlite3.connect("ai-cache.db")
    cursor = conn.cursor()
    sql = 'select query,response from cache where query like ?'
    result = cursor.execute(sql,(query,))
    result = result.fetchone()
    conn.close()
    return result
### CONTINUED ON NEXT PAGE --->>>
```

```
def ai(query):
  response: ChatResponse = chat(model='gpt-oss', messages=[
    {
      'role': 'user',
      'content': query,
    },
  ])
  return response.message.content

create()

while True:
  query = input('How Can I Help:')
  memory = find(query)
  print(memory)
  if memory:
    print(memory[0])
    print(memory[1])
  else:
    response = ai(query)
    new(query, response)
    print(query)
    print(response)
    print('-----')
```

LAB - Rules

This lab creates rules that are injected into your prompt that the LLM is supposed to follow. You are able to add rules by saying 'please;, list current rules by saying 'list', and delete rules using 'delete' and the rule number. If you don't use one of these keywords the system gets all of the current rules along with your query and that is sent to the LLM.

Note: The LLM quality matters for how cclosely the rules are followed. gpt-oss works great, buut granite4:350m generally ignores the rules.

sqlite-ollama-rules.py

```
from ollama import chat, ChatResponse
import sqlite3

class db:
    def create():
        conn = sqlite3.connect("ai-rules.db")
        cursor = conn.cursor()
        cursor.execute("""
        CREATE TABLE IF NOT EXISTS rule(
            rule
        );
        """)
        conn.commit()
        conn.close()

    def rule_new(rule):
      conn = sqlite3.connect("ai-rules.db")
      cursor = conn.cursor()
      sql = 'insert into rule(rule) values(?)'
      cursor.execute(sql,(rule,))
      conn.commit()
      conn.close()

    def rule_list():
      conn = sqlite3.connect("ai-rules.db")
      cursor = conn.cursor()
      sql = 'select rowid,rule from rule'
      result = cursor.execute(sql)
      result = result.fetchall()
      conn.close()

      rules = ''
      for value in result:
        rules += f'{value[0]} - {value[1]}\n'

      return rules

### CONTINUED ON NEXT PAGE --->>>
```

```
    def rule_delete(rule):
      conn = sqlite3.connect("ai-rules.db")
      cursor = conn.cursor()
      sql = 'delete from rule where rowid = ?'
      cursor.execute(sql,(rule,))
      conn.commit()
      conn.close()

def ai(query):
  response: ChatResponse = chat(model='gpt-oss', messages=[
    {
      'role': 'user',
      'content': query,
    },
  ])
  return response.message.content

db.create()

while True:
  query = input('How Can I Help: (please/list/delete) ')
  if 'please' in query:
    db.rule_new(query)
    rules = db.rule_list()
    print(rules)
  elif 'list' in query:
    rules = db.rule_list()
    print(rules)
  elif 'delete' in query:
    rule = query.split(' ')
    rule[1] = rule[1].strip()
    db.rule_delete(rule[1])
    rules = db.rule_list()
    print(rules)
  else:
    rules = db.rule_list()
    query = f'''
              \nFollow these rule:
              \n{rules}
              --
              \nTo Answer this query:
              \n{query}
              '''
    response = ai(query)
    print(response)
    print('-----')
```

Bottle Web App Framework

What is Bottle

Bottle is a web app framework for Python. It allows you to create Web Applications or websites using the Python language. It is only one component of a functioning platform that will require a web server, reverse proxy server and other services.

Why Use Bottle

There are many web app frameworks available for Python. Django is one of the most full featured, and Flask is a favorite for many coders. We use Bottle because it is very simple, easy to run, and robust enough for small projects.

You can learn to build for Bottle in a few hours, and you can run you web app by simply calling the Python script without requiring additional setup. Django can be seen as a far superior framework, but simply explaining it’s architecture and setup takes significant time.

Architecture

- OS = Operating System the app is running on.
- Data and Record Stores - Data Stores and Databases many times are not on the same server as the App.
- Python - The backend programming language used to build the app and interface with resources
- Bottle - The Web App Framework that turns the Python code into something that can be used in a web browser
- Gunicorn - A WSGI HTTP Web Server that provides basic services required to make the Bottle App accessible by web traffic
- NGINX - A Reverse Proxy server that provides routing, caching and other services to improve visitor experieince.
- DNS - Resolving your domain name to the IP Address of your server.

Run, Route and Return

To write a basic Bottle script you will first need to import all of the modules you will need.

For this “hello world” script we will import the route and run modules.

The route module provides you with the @route decorator. This will add the functionality for web page names. So ‘/’ will route to the root of the web app, whereas ‘/about’ would route to an about page.

Under the @route decorator in your script you create a Python function. The function names should be unique on the page.

For this example we use the return() function which will return “hello world” so that when a user goes to 127.0.0.1 in their web browser they will see “hello world”.

The run() function defines the host and port that should respond to web requests. For host you can use 127.0.0.1 for the loopback address only. You can use 0.0.0.0 which will respond to any IP address available on the computer, or you can define a specific IP address to respond to.

For host you can also use a domain name to respond to. Note that if you use ‘localhost’ some computers try to resolve the domain name through IPv6 which will add up to a 30 second lag time.

The port value should generally be set to 8080. This TCP port is the standard HTTP port other than 80. If you have multiple Bottle apps running at once they will need to use different ports.

When you run this script you will need to go to http://127.0.0.1:8080 in your web browser to see “hello world”.

```
from bottle import route, run

@route('/')
def index():
      return('hello world')

run(host='127.0.0.1',port='8080')
```

Routes

Routes in Bottle can be though of as web pages. So SiliconDojo.com/about would send the visitor to the "about" web page.

The route module provides a @route() decorator to your main function. In Python decorators add additional functionality to functions.

Static Routes

Static Routes simply route visitors to different "pages". Note that each "page" has a unique function name.

bottle-route-static.py

```
from bottle import route, run

@route('/')
def index():
      return('hello world')

@route('/about')
def about():
      return('Something cool about me')

@route('/services')
def services():
      return('We offer many services')

run(host='127.0.0.1',port='8080')
```

Dynamic Routes

Dynamic Routes allow you to send variable values within the URL. You signify the variable name with <> and then you pass the variable name within the function call.

You can set data types for variables with int, float, path and more. Note that if the data type of the value being sent does not match the function call will fail. (Putting a float in an int route will fail outright not simply remove the decimal point values.)

To create a default value for a route you can use a static route, and assign a default value in the function call.

```
@route('/')
@route('/<name>')
def index(name='NOT SET'):
      return(f'hello {name}')
```

bottle-route-dynamic.py

```
from bottle import route, run

@route('/')
@route('/<name>')
def index(name='NOT SET'):
      return(f'hello {name}')

@route('/add/<num1:int>/<num2:int>')
def add(num1,num2):
    total = num1 + num2
    return(f'{num1} plus {num2} equals {total}')

@route('/mult/<num1:float>/<num2:float>')
def multiply(num1,num2):
    total = num1 * num2
    return(f'{num1} time {num2} equals {total}')

run(host='127.0.0.1',port='8080')
```

Methods – POST and GET

With the @route() decorator you can add method=['GET','POST'] to allow POST and GET values be sent to the function.

POST

To access the POST value sent to the script you use the requests.forms.get() function.

```
VARIABLE = request.forms.get('VALUE')
```

To test for if a POST value is sent to the script you use an if statement. This allows one thing to be done if a POST value is sent, and something else if not.

```
if request.method == 'POST':
```

Note that you have to import the post() function from bottle to gain access to it.

bottle-post.py

```
from bottle import run, route, post, request

@route('/', method=['GET','POST'])
def index():
    if request.method == 'POST':
        query = request.forms.get('query')
    else:
        query = 'NO ONE'

    page = f'''
            <h1>Web App</h1>
            <form action="/" method="post">
                Name: <input type="text" name="query">
                <br>
                <input type="submit">
            </form>
            <strong>Hello {query}</strong><br>
            '''
    return page

run(host='127.0.0.1', port=8080)
```

You can also use the @post decorator on it's own. This might be useful if data is being sent to your script, but no return is expected.

```
@post('/')
def do_something():
      query = request.forms.get('query')
      # Do Something
```

GET

You can send values to your script using a GET method in the URL

```
http://127.0.0.1:8080/?color=blue
```

On a webpage you can format a hyperlink with variable values in it.

After the IP Address or Domain Name you use '?' to start the GET Method. You then set the variable name, use '=' to set the value. You can send numerous values by connecting them with '&'

```
crappy-app.com?color=blue&font=24
```

To access the GET Values in Bottle you use request.query.get()

```
VARIABLE = request.query.get('VALUE')
```

To check if there is a value beings sent through the GET method you can use an if statement.

```
if request.method == 'GET':
```

Note that for GET you do not have to import a get() function from bottle.

bottle-get.py

```
from bottle import run, route, request

@route('/', method=['GET','POST'])
def index():
    if request.method == 'GET':
        color = request.query.get('color')
    else:
        color = 'white'

    page = f'''
    <body style="background-color:{color};">
            <h1>Web App</h1>
            <a href="/?color=green">Green</a>
            <a href="/?color=red">Red</a>
            <a href="/?color=blue">Blue</a>
            '''
    return page

run(host='127.0.0.1', port=8080)
```

You can also use the @get() decorator if the incoming traffic is not expecting a response.

```
@get('/')
def do_something():
    query = request.query.get('query')
    # Do Something
```

Static Files

For security reasons it is considered best not to host static files on the same server that the Bottle App is running on.

You can use the static_file() function to be able to serve files from a local directory. These may be image or multimedia files or CSS style sheets and JavaScript files.

You can have any name for the folder for the static files. The route path will be use a Dynamic Route with the filename as the variable value and a data type of path.

In the static_file() function for the root= setting use ./foldername or the full path to the folder.

In the following script we have created a folder called static in the current working directory, and placed a png picture of a robot in it.

bottle-static.py

```
from bottle import run, route, static_file

@route('/')
def index():
    page = '''
            <h1>Static Image File</h1>
            <img src="./static/robot.png">
            '''
    return(page)

@route('/static/<filepath:path>')
def server_static(filepath):
    return static_file(filepath, root='./static')

run(host='127.0.0.1', port='8080')
```

Templates

Bottle uses a Template Engine called SimpleTemplate. Templates in web app frameworks allow you to create the structure of a web page and then fill in dynamic data with variable values. This creates easily reusable pages, and also offers a level of protection agains injection attacks. Templates sanitize variable values when they are presented to stop scripts and frames from being added through variable values.

To use templates you will need to create a folder names **views** in your working directory.

When you create template files you will save with the .tpl extension.

Template files will be written with a combination of standard HTML, CSS, JavaScript and the Template Language

bottle-template.py

```
from bottle import run, route, template

@route('/')
def index():
    return template('index')

run(host='127.0.0.1', port='8080')
```

views/index.tpl

```
<h1>hello world</h1>
```

Template Variables

You can send variable values to the template and then display them using {{ variable_name }}.

Notice how this automatically encodes the injected hyperlink so that it does not work.

Intro to Stuff

bob
sue
<a href="http://cnn.com">tim</a>
pam

this is a message

Note: Adding a '!' to a variable embed will stop the sanitization. {{ !variable_name }}

bottle-template-variables.py

```
from bottle import run, route, template

@route('/')
def index():
    class_name = 'Intro to Stuff'
    roster = ['bob',
            'sue',
            '<a href="http://cnn.com">tim</a>',
            'pam']
    return template('variables', kids=roster, title=class_name,message='this is a
message')

run(host='127.0.0.1', port='8080')
```

views/variables.tpl

```
<h1>{{title}}</h1>

% for x in kids:
    {{x}} <br>
% end

<h3>{{message}}</h3>
```

Template Controls

Within a Template you can use Controls to determine how a variable value should be processed.

You begin a control with % and the control. You must end the control with **% end**.

To create and set new variables within a template you use **% variable_name.**

bottle-template-vaiables-control.py

```
from bottle import run, route, template

@route('/')
def index():
    roster = [{'name':'bob','age':13},
            {'name':'sue','age':8},
            {'name':'pam','age':22},
            {'name':'tim','age':15},
            {'name':'kim','age':19}]
    return template('control', roster=roster)

run(host='127.0.0.1', port='8080')
```

views/control.tpl

```
% for x in roster:
    {{x}} <br>
% end

% for x in roster:
    {{x['name']}} -- {{x['age']}} <br>
% end

% for x in roster:
    % if x['age'] >= 18:
        % status = 'adult'
    % elif x['age'] >= 13 and x['age'] < 18:
        % status = 'teenager'
    % else:
        % status = 'child'
    % end
    {{x['name']}} -- {{x['age']}} -- {{ status }}<br>
% end
```

LAB - Basic LLM App

This labs creates a very basic AI Web App. We create an HTML form and when the index() receives a POST value it sends that to the ai() function. The original query, and the response is then printed in HTML.

Note: Notice how the response value is formatted using P tags. Try swapping out with PRE tags. How would you use code to make the format look better?

Test: Add a hyper link in the query box and notice how when the query is printed on the output that it is a fully clickable link.

<a href="http://cnn.com">this is a query</a>

bottle-ollama.py

```
from bottle import run, route, post, request, template
from ollama import chat, ChatResponse

def ai(query):
    injection = 'Answer in fewer than 50 words'
    query = f'{injection} -- {query}'
    response: ChatResponse = chat(model='granite4:350m', messages=[
    {
        'role': 'user',
        'content': query,
    },
    ])
    return response.message.content

@route('/', method=['GET','POST'])
def index():
    if request.method == 'POST':
        query = request.forms.get('query')
        response = ai(query)
    else:
        query = '****'
        response = '****'

    page = f'''
            <h1>Web App</h1>
            <form action="/" method="post">
                Query <input type="text" name="query">
                <br>
                <input type="submit">
            </form>
            <strong>{query}</strong><br>
            <p>{response}</p>
            '''
    return page

run(host='127.0.0.1', port=8080)
```

LAB - AI Web App Using Templates

With this lab we present the web page using a template. This will sanitize the data being presented so that users cannot add an injection attack.

Try the a href injection again and notice the results.

bottle-ollama-template.py

```
from bottle import run, route, post, request, template
from ollama import chat, ChatResponse

def ai(query):
    injection = 'Answer in fewer than 50 words'
    query = f'{injection} -- {query}'
    response: ChatResponse = chat(model='granite4:350m', messages=[
    {
        'role': 'user',
        'content': query,
    },
    ])
    return response.message.content

@route('/', method=['GET','POST'])
def index():
    if request.method == 'POST':
        query = request.forms.get('query')
        response = ai(query)
    else:
        query = '****'
        response = '****'

    return template('bottle-ollama',query=query, response=response)

run(host='127.0.0.1', port=8080)
```

views/bottle-template.py

```
<h1>AI App</h1>
<form action="/" method="post">
    Query <input type="text" name="query">
    <br>
    <input type="submit">
</form>
<strong>{{ query }}</strong><br>
<p>{{ response }}</p>
```

LAB - AI Web App with SQL Caching

With this lab we are going to add caching using SQLite. When a request is made for the first time it will be sent through the ai() function and the LLM will respond. When you make an exactly the same request in the future the response will be pulled from the SQLite database.

We are reusing the ai-cache.db database that was made during the SQL portion of the course. Do notice that the create() function is called right before the run() function. This way is the database and table do not exist they will be created.

We are also reusing the template file from the last lab.

bottle-ollama-sql.py

```
from bottle import run, route, post, request, template
from ollama import chat, ChatResponse
import sqlite3

def create():
    conn = sqlite3.connect("ai-cache.db")
    cursor = conn.cursor()
    cursor.execute("""
    CREATE TABLE IF NOT EXISTS cache (
        query,
        response
    );
    """)
    conn.commit()
    conn.close()

def new(query,response):
    conn = sqlite3.connect("ai-cache.db")
    cursor = conn.cursor()
    sql = 'insert into cache(query, response) values(?,?)'
    cursor.execute(sql,(query, response))
    conn.commit()
    conn.close()

def find(query):
    conn = sqlite3.connect("ai-cache.db")
    cursor = conn.cursor()
    sql = 'select query,response from cache where query like ?'
    result = cursor.execute(sql,(query,))
    result = result.fetchone()
    conn.close()
    return result

### CONTINUED ON NEXT PAGE --->>>
```

```
def ai(query):
    injection = 'Answer in fewer than 50 words'
    query = f'{injection} -- {query}'
    response: ChatResponse = chat(model='gpt-oss', messages=[
    {
        'role': 'user',
        'content': query,
    },
    ])
    return response.message.content

@route('/', method=['GET','POST'])
def index():
    if request.method == 'POST':
        query = request.forms.get('query')
        memory = find(query)
        if memory:
            response = memory[1]
        else:
            response = ai(query)
            new(query, response)
            print(query)
            print(response)
    else:
        query = '****'
        response = '****'

    return template('bottle-ollama',query=query, response=response)

create()
run(host='127.0.0.1', port=8080)
```

Deploying A Bottle Web App

To deploy a Bottle Web App you'll need a domain name, and a VPS.

I recommend CloudFlare for Domain Names becasue SSL is built in, they have a number of tools, and they are inexpensive.

I recommend DigitalOcean for VPS's simply because they are well known, their prices are reasonable, and they're reliable.

The Basic Traffic Flow:

- Visitor Resolves Domain Name to IP Address through CloudFlare
- Visitor Routed to Your Ubuntu VPS on DigitalOcean
- NGINX Acts as a Reverse Proxy Server to Route Visitor to the WSGI HTTP Server(Gunicorn)
- Gunicorn Presents Your Bottle Web App

Basic Steps:

- Buy VPS
- Install Ubuntu Linux and Update
- Write the Bottle Web App Code
- Install Gunicorn
- Configure a Linux Service to Start Gunicorn and Point it at the Bottle Script
- Install NGINX
- Configure NGINX to route traffic to a Domain Name to the Gunicorn Service
- Buy DNS Name
- Configure Domain Name @ and www records to Point to VPS IP Address

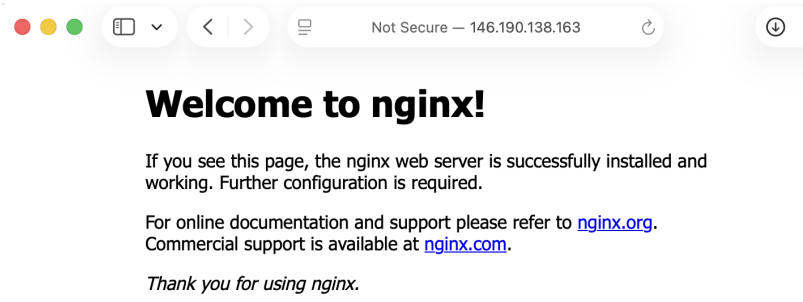
Install Requirements to Run Bottle and Verify

We will create the VPS in DigitalOcean and then use the console to connect to it.

```
sudo apt update
sudo apt upgrade
sudo apt install nginx
```

Find the VPS IP Address in the Digital Ocean Console.

Go to that address in your web browser. You should receive a "Welcome to nginx!" page.



Install PIP and VENV

```
sudo apt install python3-pip
sudo apt install python3-venv
```

Create Folder, Environment and Script for Web App

```
cd /var/www
mkdir webapp
cd webapp

sudo python3 -m venv venv
source venv/bin/activate

python3 -m pip install bottle

nano webapp.py
```

In the webappy.py script we will set host to 0.0.0.0 so that all traffic that goes to the server at port 8080 will be responded to by the Bottle App.

webapp.py

```
from bottle import run, route

@route('/')
def index():
        return('Hello World')

run(host='0.0.0.0', port='8080')
```

Start the Bottle Script

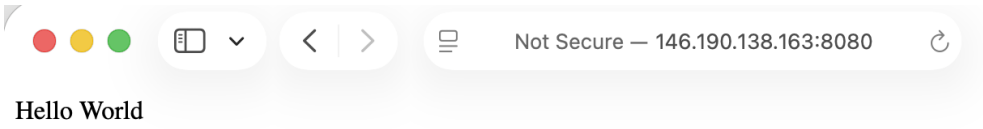
```
python3 webapp.py
```

Find You IP Address on Digital Ocean Console.

In your web browser go to:

```
http://your.ip.address:8080
```

Note: It may take a minute for the app to initially respond.



Install Gunicorn

Gunicorn is a WSGI HTTP Server that is designed to handle visitor traffic. The most important component is that it is multi-threaded so that multiple visitor requests can be handled at once.

You will need to modify your webapp.py script to put the run() function under an if statement. This means that run() will only trigger if you call the script from the command line, it will not trigger when Gunicorn uses the script.

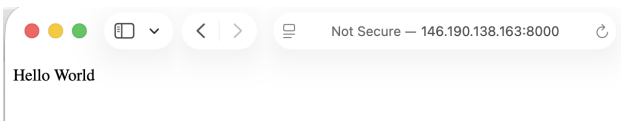
You will also add the default_app() function to give your script an app name that can be called with Gunicorn.

```
sudo apt install gunicorn
```

```
from bottle import route, run, default_app

@route('/')
def index():
        return('Hello World')

if __name__=='__main__':
        run(host='0.0.0.0', port='8080')

myapp = default_app()
```

At the command line run this command and then verify if "hello world" can be accessed from another computer using the IP address of your VPS.

```
gunicorn --workers 4 --bind 0.0.0.0:8000 webapp:app
```



Create Linux Service

Creating a service will allow Ubuntu to start the web app when the system reboots, and ill allow you to restart and stop the app as a normal service while the system in running.

Create the service file using Nano.

```
sudo nano /etc/systemd/system/webapp.service
```

```
[Unit]
Description=My Cool App
After=network.target

[Service]
User=www-data
Group=www-data
WorkingDirectory=/var/www/webapp
Enviornment="PATH=/var/www/webapp/venv/bin"
ExecStart=/var/www/webapp/venv/bin/gunicorn --workers 4 --bind 127.0.0.1:8000
webapp:app

[Install]
WantedBy=multi-user.target
```

Reload Services, Start webapp Service, Set webapp to Start when OS Reboots

```
sudo systemctl daemon-reload
sudo systemctl start webapp
sudo systemctl enable webapp
```

Configure NGINX to Route to Gunicorn

NGINX is a reverse proxy server. It is used to route visitor traffic and provide other services. Your visitors will initially hit NGINX and then it will route them to Gunicorn.

We will set the default route to go to Gunicorn for simplicity.

```
sudo rm /etc/nginx/sites-available/default
```

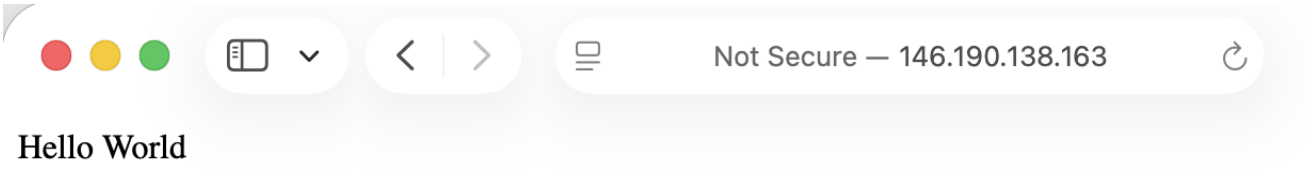
```
sudo nano /etc/nginx/sites-available/default
```

```
server {
    listen 80 default_server;
    server_name 127.0.0.1;

    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Verify NGINX Configuration Works and Restart Service

```
sudo nginx -t
sudo systemctl restart nginx
```

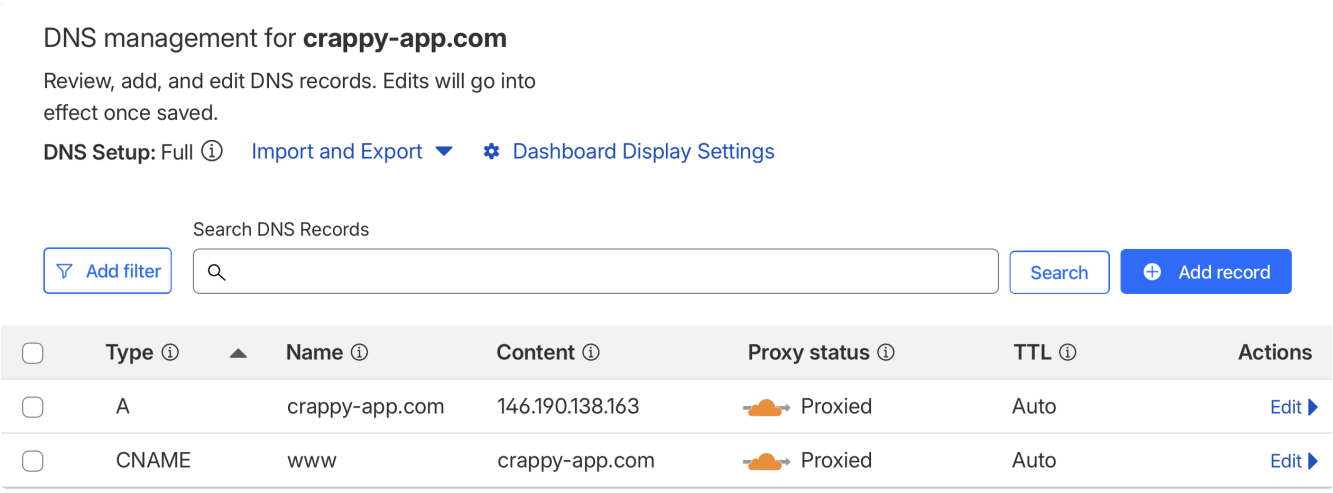
Check the VPS IP Address Routes to Web App



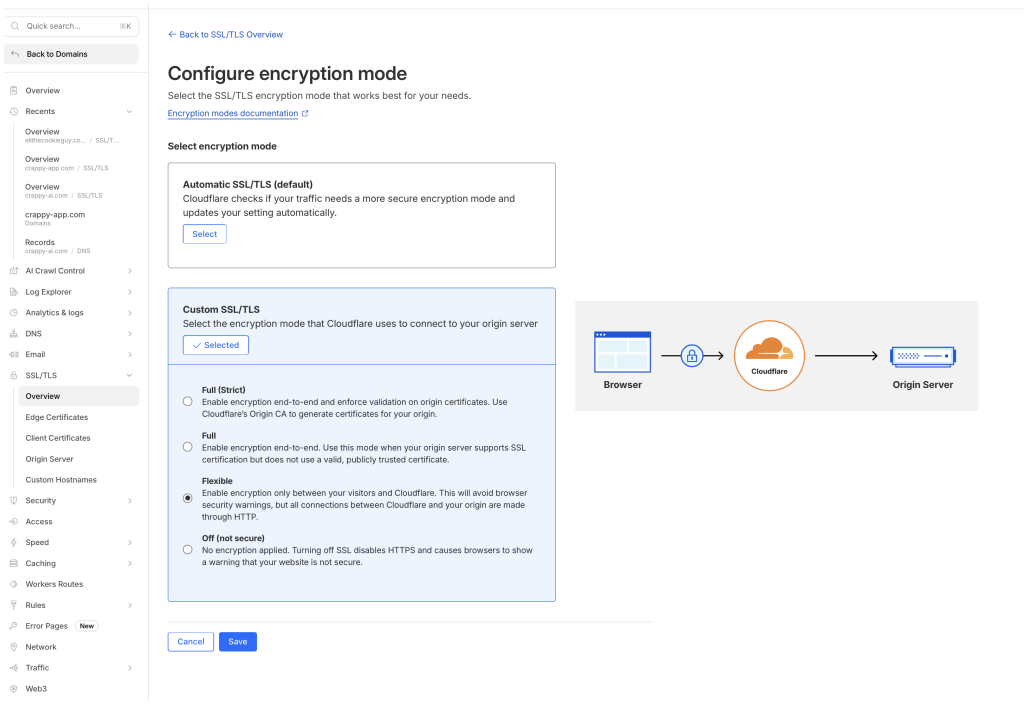
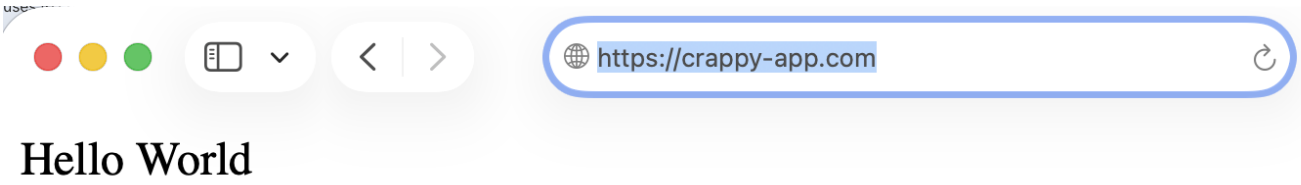
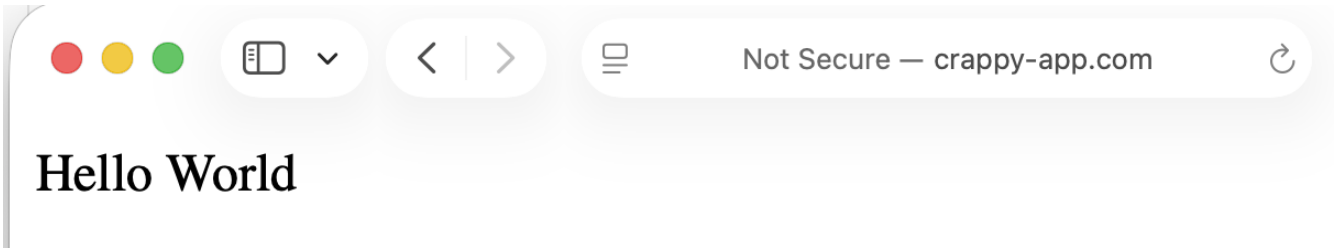
Set DNS to Point to VPS

Now to make your domain name point to your server you need to configure DNS.

- Go to CloudFlare or your DNS Registrar.
- Set A Record to point to the VPS IP Address.
- Set CNAME record for www to point to @.



Under SSL/TLS Set Encryption Mode to **Flexible**



Under both HTTPS and HTTP you should now be able to reach your web app.